

Calcolabilità e complessità

Alessandro Carella

Indice

1	Automi e linguaggi	2
1.1	Automi finiti	2
1.2	Non determinismo	6

Capitolo 1

Automi e linguaggi

1.1 Automi finiti

Gli automi finiti sono un buon modello per computer una quantità estremamente di memoria che permette di fare molte cose utili. Il sistema di controllo per una porta automatica è un esempio di tale dispositivo.

Gli automi finiti e la loro controparte probabilistica, le *catene di Markov* sono utili strumenti quando cerchiamo di riconoscere pattern in dati. Questi dispositivi sono usati nell'elaborazione vocale e nel riconoscimento ottico dei caratteri. Le catene di Markov sono state perfino usate per modellare e predire i cambiamenti dei prezzi nei mercati finanziari.

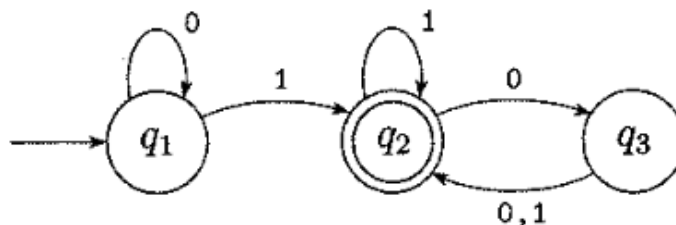
Definizione formale di automa finito

DEFINIZIONE 1.1: AUTOMA FINITO

Un *automa finito* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove:

1. Q è un insieme finito chiamato l'insieme degli *stati*;
2. Σ è un insieme finito chiamato *alfabeto*;
3. $\delta : Q \times \Sigma \rightarrow Q$ è la *funzione di transizione*;
4. $q_0 \in Q$ è lo *stato iniziale*;
5. $F \subseteq Q$ è l'*insieme degli stati accettanti*.

Consideriamo il seguente automa M_1 :



Possiamo descrivere M_1 formalmente ponendo $M_1 = (Q, \Sigma, \delta, q_1, F)$ ove:

1. $Q = \{q_1, q_2, q_3\}$;
2. $\Sigma = \{0, 1\}$;
3. δ è descritta come segue:

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 è lo stato iniziale;
5. $F = \{q_2\}$

Se A è l'insieme di tutte le stringhe che la macchina M accetta, diciamo che A è il *linguaggio della macchina* M e scriviamo $L(M) = A$. Diciamo che M *riconosce* A o che M *accetta* A . Poichè il termine *accetta* ha significati diversi quando ci riferiamo a macchine che accettano stringhe e macchine che accettano linguaggi, per evitare confusione preferiamo per i linguaggi il termine *riconosce*.

Una macchina può accettare diverse stringhe, ma riconosce sempre soltanto un linguaggio. Se la macchina non accetta alcuna stringa, essa riconosce ancora un linguaggio, il linguaggio vuoto $\emptyset$.

Nel nostro esempio, sia:

$$A = \{w \mid w \text{ contiene almeno un } 1 \text{ e un numero pari di } 0 \text{ segue l'ultimo } 1\}$$

Definizione formale di computazione

Finora abbiamo descritto gli automi finiti informalmente, usando diagrammi di stato, e con una definizione formale, come una quintupla. La descrizione informale è più facile da comprendere all'inizio, ma la definizione formale è più utile per rendere la notazione precisa, risolvendo ogni ambiguità.

Facciamo lo stesso per la computazione di un automa finito.

Sia $M = (Q, \Sigma, \delta, q_0, F)$ un automa finito e sia $w = w_1 w_2 \cdots w_n$ una stringa dove ogni w_i è un elemento dell'alfabeto Σ . Allora M accetta w se esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$;
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ per $i = 0, \dots, n-1$;
3. $r_n \in F$.

La prima condizione afferma che la macchina inizia nello stato iniziale. La seconda condizione afferma che la macchina passa da stato a stato in base alla funzione di transizione. La terza condizione afferma che la macchina accetta il suo input se termina la lettura in uno stato accettante. Diciamo che M riconosce il linguaggio A se $A = \{w \mid M \text{ accetta } w\}$.

DEFINIZIONE 1.2: LINGUAGGIO REGOLARE

Un linguaggio è chiamato un *linguaggio regolare* se un automa finito lo riconosce.

Le operazioni regolari

Dopo aver introdotto e definito gli automi finiti e i linguaggi regolari, iniziamo a studiarne le proprietà. In aritmetica, gli oggetti di base sono i numeri e gli strumenti sono le operazioni per trattarli, come $+$ e $\times$. Nella teoria della computazione, gli oggetti sono i linguaggi e gli strumenti includono operazioni specificatamente progettate per trattarli. Definiamo tre operazioni sui linguaggi, chiamate *operazioni regolari*, e le usiamo per studiare le proprietà dei linguaggi regolari.

DEFINIZIONE 1.3: OPERAZIONI REGOLARI

Siano A e B linguaggi. Definiamo le operazioni regolari *unione*, *concatenazione* e *star* come segue:

- **Unione:** $A \cup B = \{x \mid x \in A \text{ o } x \in B\}$
- **Concatenazione:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$

- **Star:** $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ e ogni } x_i \in A\}$

Esempio

Sia Σ l'alfabeto standard a 26 lettere $\{a, b, \dots, z\}$. Se $A = \{good, bad\}$ e $B = \{boy, girl\}$, allora:

$$A \cup B = \{good, bad, boy, girl\}$$

$$A \circ B = \{goodboy, goodgirl, badboy, badgirl\}$$

$$A^* = \{\epsilon, good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, \dots\}$$

Sia $\mathbb{N} = \{1, 2, 3, \dots\}$ l'insieme dei numeri naturali. Quando diciamo che $\mathbb{N}$ è *chiuso rispetto alla moltiplicazione*, intendiamo che per ogni x e per ogni y in $\mathbb{N}$, il prodotto $x \times y \in \mathbb{N}$. Invece $\mathbb{N}$ non è chiuso rispetto alla divisione, poichè 1 e 2 sono in $\mathbb{N}$ ma $1/2 \notin \mathbb{N}$.

In generale, una classe di oggetti è *chiusa* rispetto a un'operazione se l'applicazione di questa operazioni a elementi della classe restituisce un oggetto ancora nella classe. Mostriamo che la classe dei linguaggi regolari è chiusa rispetto a tutte e tre le operazioni regolari.

Teorema 1.1: Chiusura unione

La classe dei linguaggi regolari è chiusa rispetto all'operazioni di unione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, lo è anche $A_1 \cup A_2$.

Quindi, abbiamo due linguaggi regolari, A_1 e A_2 , e vogliamo mostrare che anche $A_1 \cup A_2$ è regolare. Poichè A_1 e A_2 sono regolari, sappiamo che sono riconosciuti da degli automi finiti. Quindi per provare che $A_1 \cup A_2$ è regolare, dobbiamo esibire un automa finito, M , che riconosce $A_1 \cup A_2$.

Dimostrazione. Supponiamo che M_1 riconosca A_1 , dove $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ ed M_2 riconosca A_2 , dove $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$. Costruiamo M che riconosce $A_1 \cup A_2$ dove $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ e } r_2 \in Q_2\}$.
Questo insieme è il *prodotto cartesiano* degli insiemi Q_1 e Q_2 ed è denotato con $Q_1 \times Q_2$.
2. Σ , l'alfabeto è lo stesso di M_1 e M_2 . In questo teorema e in quelli successivi, assumiamo per semplicità che sia M_1 che M_2 abbiano stesso alfabeto di simboli. Tuttavia, se così non fosse, $\Sigma = \Sigma_1 \cup \Sigma_2$.

3. δ , la funzione di transizione, è definita come segue. Per ogni $(r_1, r_2) \in Q$ e ogni $a \in \Sigma$, sia

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

4. q_0 è la coppia (q_1, q_2) .
5. F è l'insieme delle coppie in cui l'uno o l'altro elemento è uno stato accettante di M_1 o M_2 . Possiamo scriverlo come

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ o } r_2 \in F_2\}$$

Questa espressione è uguale a $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

□

Abbiamo mostrato che l'unione di due linguaggi regolari è regolare, provando che la classe dei linguaggi regolari è chiusa rispetto all'operazione di unione. Ora mostriamo che la classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

Teorema 1.2: Chiusura concatenazione

La classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, allora lo è anche $A_1 \circ A_2$.

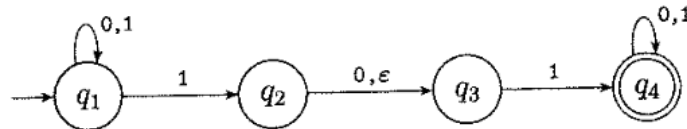
Per provare questo teorema, possiamo iniziare con gli automi finiti M_1 ed M_2 che riconoscono i linguaggi regolari A_1 ed A_2 . Ma ora, invece di costruire M che accetta l'input se M_1 o M_2 lo accetta, esso deve accettare se il suo input può essere diviso in due parti, tali che M_1 accetta la prima parte ed M_2 accetta la seconda parte. Il problema è che M non sa dividere il suo input. Per risolvere questo problema, introduciamo una tecnica chiamata *non determinismo*.

1.2 Non determinismo

Negli esempi visti precedentemente, ogni passo di una computazione seguiva univocamente dal passo precedente. Quando la macchina è in un dato stato e legge il simbolo di input successivo, sappiamo qual è lo stato successivo, esso è univocamente determinato. In questo caso parliamo di computazione *deterministica*. In una macchina *non deterministica*, possono esistere diverse scelte per lo

stato successivo in ogni punto.

Il non determinismo è una generalizzazione del determinismo, quindi ogni automa finito deterministico è automaticamente un automa finito non deterministico. Consideriamo il seguente automa:



La differenza tra un automa finito deterministico, DFA, e un automa finito non deterministico, indicato con NFA, è evidente. In primo luogo, ogni stato di un DFA ha sempre esattamente un arco uscente per ogni simbolo nell'alfabeto. L'NFA viola questa regola, infatti, lo stato q_1 dell'automata della figura precedente, ha un arco uscente per 0, ma ne ha due per 1. In un NFA, uno stato può avere zero, uno, o più archi uscenti per ogni simbolo dell'alfabeto.

Inoltre, in un DFA, le etichette sugli archi di transizione sono simboli dell'alfabeto. Questo NFA ha un arco con etichetta ϵ .

Durante la computazione di un NFA, quando si giunge ad uno stato con più modi per procedere, la macchina si divide in più copie di sè stessa e segue tutte le possibilità in parallelo. Infine, se una qualunque di queste copie della macchina è in uno stato accettante alla fine dell'input, l'NFA accetta la stringa di input.

Il non determinismo può essere visto come una specie di computazione parallela dove "processi" multipli indipendenti o "threads" possono essere eseguiti contemporaneamente. Un altro modo per considerare una computazione non deterministica è vederla come un albero di possibilità. La radice dell'albero corrisponde all'inizio della computazione. Considerando sempre l'automata precedente, l'albero di computazione è il seguente:

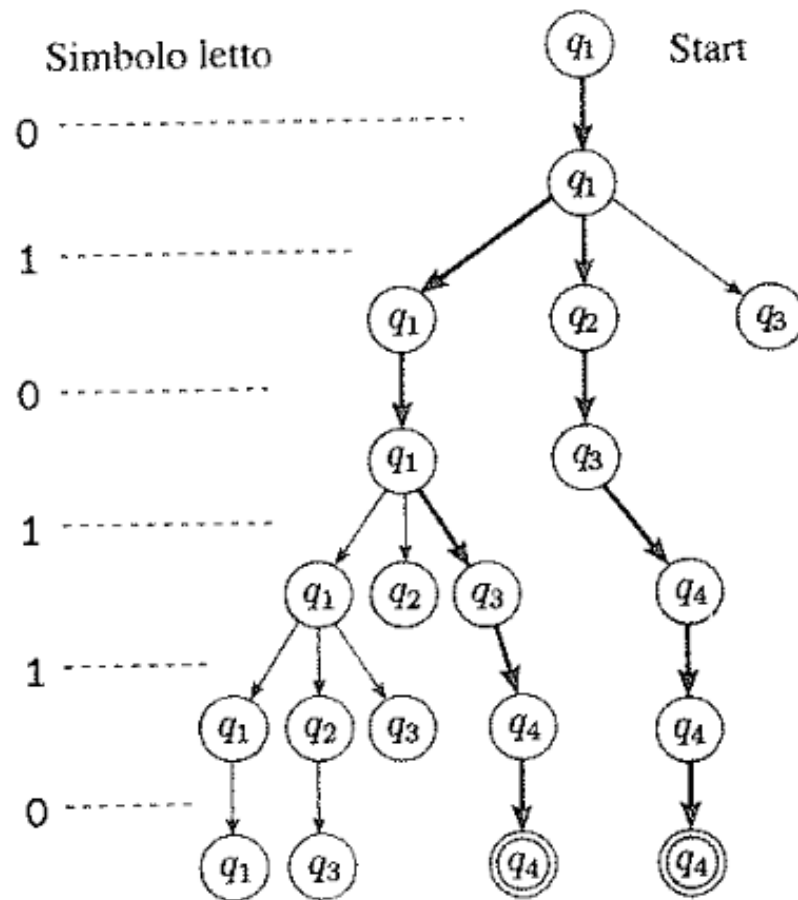


Figura 1.1: La computazione dell'automa precedente su 010110

Definizione formale di automa finito non deterministico

La definizione formale di automa finito non deterministico è simile a quella di automa finito deterministico. Essi differiscono nel tipo di funzione di transizione. In un DFA, la funzione di transizione prende uno stato e un simbolo di input e produce lo stato successivo. In un NFA, la funzione di transizione prende uno stato e un simbolo di input o *la stringa vuota* e produce l'*insieme dei possibili stati successivi*.

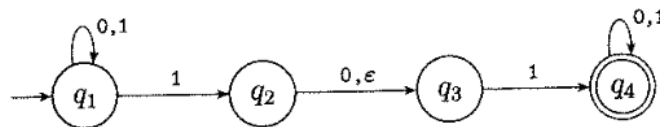
DEFINIZIONE 1.4: AUTOMA FINITO NON DETERMINISTICO

Un *automa finito non deterministico* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove

1. Q è un insieme finito di stati,
2. Σ è un alfabeto finito,
3. $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ è la funzione di transizione,
4. $q_0 \in Q$ è lo stato iniziale,
5. $F \subseteq Q$ è l'insieme degli stati di accettazione.

Esempio

Riconsideriamo il seguente NFA N_1 :



La descrizione formale di N_1 è $(Q, \Sigma, \delta, q_1, F)$ dove

1. $Q = \{q_1, q_2, q_3, q_4\}$
2. $\Sigma = \{0, 1\}$
3. δ è data da

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 è lo stato iniziale
5. $F = \{q_4\}$

La definizione formale di computazione per un NFA è simile a quella per un DFA. Sia $N = (Q, \Sigma, \delta, q_0, F)$ un NFA e sia w una stringa sull'alfabeto Σ . Diciamo

che N accetta w se possiamo scrivere w come $w = y_1 y_2 \dots y_m$, dove ciascun y_i è un elemento di $\sum_\epsilon$ ed esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$ per $i = 0, \dots, m-1$
3. $r_m \in F$

Equivalenza tra gli NFA e i DFA Gli automi finiti deterministici e quelli non deterministici riconoscono la stessa classe di linguaggi. Diciamo che due macchine sono *equivalenti* se esse riconoscono lo stesso linguaggio.

Teorema 1.3: Equivalenza NFA e DFA

Per ogni automa finito non deterministico esiste un automa finito deterministico equivalente.

Se un linguaggio è riconosciuto da un NFA, allora dobbiamo mostrare l'esistenza di un DFA che lo riconosce. L'idea è trasformare l'NFA in un DFA equivalente che simula l'NFA.

Dimostrazione. Sia $N = (Q, \sum, \delta, q_0, F)$ l'NFA che riconosce un linguaggio A . Costruiamo un DFA $M = (Q', \sum, \delta', q'_0, F')$ che riconosce A . Prima di fare la costruzione completa, consideriamo inizialmente il caso più semplice in cui N non ha ϵ -archi.

1. $Q' = \mathcal{P}(Q)$
Ogni stato di M è un insieme di stati di N .
2. Per $R \in Q'$ e $a \in \sum$, sia $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a)\}$ per qualche $r \in R$
Se R è uno stato di M , esso è anche un insieme di stati di N . Quando M legge un simbolo a nello stato R . Poichè da ogni stato si può andare in un insieme di stati, prendiamo l'unione di tutti questi insiemi. Un altro modo per scrivere quest'espressione è:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

3. $q'_0 = \{q_0\}$

4. $F' = \{R \in Q' \mid R \text{ contiene uno stato accettante di } N\}$

Ora dobbiamo considerare gli ϵ -archi. Per farlo introduciamo qualche ulteriore notazione. Per ogni stato R di M , definiamo $E(R)$ come la collezione di stati che possono essere raggiunti dagli elementi di R proseguendo solo con ϵ -archi, includendo gli stessi elementi di R . Formalmente, per $R \subseteq Q$ sia

$$E(R) = \{q \mid q \text{ può essere raggiunto da } R \text{ attraverso 0 o più } \epsilon\text{-archi}\}$$

Modifichiamo la funzione di transizione di M ponendo le dita su tutti gli stati che possono essere raggiunti proseguendo attraverso ϵ -archi dopo ogni passo. Sostituendo $\delta(r, a)$ con $E(\delta(r, a))$ realizziamo questo effetto. La funzione di transizione è la seguente.

$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ per qualche } r \in R\}$$

Inoltre, dobbiamo modificare lo stato iniziale di M per muovere inizialmente le dita su tutti i possibili stati che possono essere raggiunti dallo stato iniziale di N attraverso gli ϵ -archi. Cambiare q'_0 in $E(\{q_0\})$ realizza questo risultato. Abbiamo così completato la costruzione del DFA M che simula l'NFA N . $\square$

Corollario 1.1

Un linguaggio è regolare se e solo se qualche automa finito non deterministico lo riconosce.

Una direzione della condizione “se e solo se” afferma che un linguaggio è regolare se qualche NFA lo riconosce. Tuttavia, abbiamo dimostrato che ogni NFA può essere trasformato in un DFA. Conseguentemente, se un NFA riconosce un linguaggio, lo fa anche qualche DFA, e quindi il linguaggio regolare. L'altra direzione della condizione “se e solo se” afferma che un linguaggio è regolare solo se qualche NFA lo riconosce. Questa condizione è vera perchè un linguaggio regolare ha un DFA che lo riconosce e un DFA è anche un NFA.

Chiusura rispetto alle operazioni regolari

Torniamo alla chiusura della classe dei linguaggi regolari rispetto alle operazioni regolari. Dobbiamo provare che l'unione, la concatenazione e lo star di linguaggi regolari sono ancora regolari.

Teorema 1.4

La classe dei linguaggi regolari è chiusa rispetto all'operazione di unione.

Abbiamo i linguaggi regolari A_1 ed A_2 e vogliamo provare che $A_1 \cup A_2$ è regolare. L'idea è provare che due NFA, N_1 ed N_2 per A_1 ed A_2 , e comporli in un nuovo NFA, N . La macchina N deve accettare il suo input se N_1 o N_2 accetta questo input. La nuova macchina ha uno stato iniziale che si dirama negli stati iniziali delle vecchie macchine con ϵ archi.

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere $A_1 \cup A_2$.

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$
2. Lo stato q_0 è lo stato iniziale di N
3. L'insieme degli stati accettanti è $F = F_1 \cup F_2$
4. Definiamo δ in modo che per ogni $q \in Q$ e $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

□

Teorema 1.5: L

classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

L'idea è prendere due NFA, N_1 ed N_2 , per A_1 ed A_2 , e combinarli in un nuovo NFA N . Poniamo come stato iniziale di N lo stato iniziale N_1 . Gli stati accettanti di N_1 hanno degli ulteriori ϵ -archi che non deterministicamente permettono di diramarsi in N_2 ogni volta che N_1 è in uno stato accettore, indicando che ha trovato un pezzo iniziale dell'input che costituisce una stringa in A_1 . Gli stati accettandi di N sono gli stati accettanti di N_2 .

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_1, F_2)$ per riconoscere $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$

2. Lo stato q_1 è uguale allo stato iniziale di N_1
3. Gli stati accettanti F_2 sono uguale agli stati accettanti di N_2
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \sum_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ e } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

□

Teorema 1.6

La classe dei linguaggi regolari è chiusa rispetto all'operazione star.

Prendiamo un NFA N_1 per A_1 e lo modifichiamo per riconoscere A_1^* . L'NFA N risultante accetterà il suo input quando esso può essere diviso in varie parti ed N_1 accetta ogni parte. Possiamo costruire N come N_1 con ϵ -archi supplementari che dagli stati accettanti ritornano allo stato iniziale. In questo modo, quando l'elaborazione giunge alla fine di una parte che N_1 accetta, la macchina N ha la scelta di tornare indietro allo stato iniziale per provare a leggere un'altra parte che N_1 accetta. Inoltre, dobbiamo modificare N in modo che accetti ϵ , che è sempre un elemento di A_1^* .

Dimostrazione. Sia $N_1 = (Q_1, \sum, \delta_1, q_1, F_1)$ che riconosce A_1 . Costruiamo $N = (Q, \sum, \delta, q_0, F)$ per riconoscere A_1^* .

1. $Q = \{q_0\} \cup Q_1$
2. Lo stato q_0 è il nuovo stato iniziale.
3. $F = \{q_0\} \cup F_1$
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \sum_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(qa) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ e } a = \epsilon \\ \{q_1\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

